

Software Cost Metrics and Estimation

Software Cost Estimation

- La stima dei costi del software è il processo di previsione della quantità di impegno richiesto per creare un sistema software e del tempo necessario per svilupparlo.
- I modelli forniscono algoritmi matematici per calcolare il costo in funzione di un numero di variabili come la dimensione (LOC, Function Points, COSMIC) e/o la complessità (complessità ciclomatica, ecc.)

Techniques

- Expert judgement
- Algorithmic models
 - Cost models (e.g., COCOMO, COCOMO II)
 - Constraint models (e.g., SLIM)
- Analogy
 - Machine learning (e.g., Case-based Reasoning, Classification Systems, etc.)
 - Etc.
- Other
 - Top-dow estimation
 - Bottom-up estimation
 - Etc.

Expert judgement

- Uno o più esperti dell'azienda (nello sviluppo del software così come del dominio dell'applicazione) sulla base della loro esperienza forniscono previsioni dei costi.
- Il processo si ripete fino a raggiungere un consenso (e.g., tramite metodo Delphi).
- Vantaggio: relativamente economico. Accurato quando gli esperti hanno una notevole esperienza con sistemi simili.
- Svantaggio: molto impreciso se non ci sono «esperti»! Difficile individuare le ragioni in caso di stima non accurata.

Analogy

- Applicabile quando in passato sono stati eseguiti altri progetti nello stesso dominio dell'applicazione.
- Il costo di un nuovo progetto viene individuato confrontando il progetto con uno o più progetti (già completati) simili nello stesso dominio dell'applicazione.
- Vantaggi: accurato se sono disponibili progetti simili
- Svantaggi: impossibile se non è disponibile alcun progetto comparabile. Richiede un repository di progetti pregressi. Difficoltà ad individuare la funzione di somiglianza.

Analogy (Case-Based Reasoning)

Effort=f(Size)

Attributes	New Case	Retrieved Case 1	Retrieved Case 2
Project Category	Real Time	Real Time	Simulator
Language	C++	C++	C++
Team Size	10	10	9
System Size	150	200	175
Effort	?	1000	950
Similarity		90%	~50%

Strategia per predire Effort:

- Adattare effort del progetto
- Media effort dei 2 progetti
- Media pesata dei 2 progetti

Regole di adattamento possibili:

- Predetto= $150/200*1000$
- Predetto= $[(150/200*1000)+(150/175*950)]/2$
- Predetto= $(150/200*1000)*9/14 + (150/175*950)*5/14$

Algorithmic models

- Utilizza le informazioni sui costi storici e le mette in relazione con alcune misure del progetto software.
- Due categorie principali (Kitchenham, 1991):
 - Cost models: fornire stime del costo del progetto (impegno) e pianificazione in base alle dimensioni; derivato da dati empirici.
 - Constraint models: si considera una relazione più complessa tra costo (impegno), schedulazione e dimensione, i.e., la produttività non solo in funzione dello sforzo (input) e della dimensioni (output) ma anche condizionata dalla pianificazione e / o dalla complessità.

Cost models

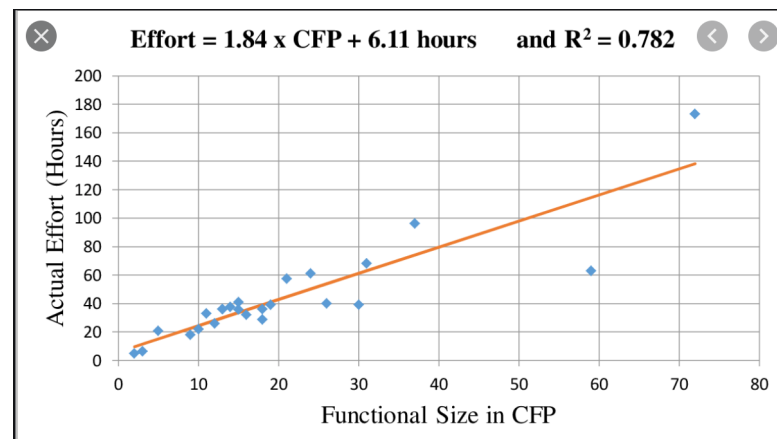
- I modelli forniscono stime dirette dello sforzo.
- I modelli in genere si basano su:
 - un cost factor primario come la dimensione
 - una serie di fattori di aggiustamento secondari o cost driver
- I cost driver possono essere caratteristiche del progetto, processo, prodotti o risorse che influenzano lo sforzo.
- I cost driver sono utilizzati per adeguare la stima preliminare fornita dal cost driver primario.

Cost models

- Effort vs Size (e.g., LOC, FP, o CFP) per una serie di progetti.
- La linea di migliore adattamento («best fit») viene determinata per le osservazioni (punti).
 - Se la Size fosse un predittore «perfetto» dell'Effort, ogni punto del grafico si troverebbe sulla linea di «best fit».
- Nella realtà, c'è un errore residuo significativo. E' necessario quindi determinare i fattori che causano variazione tra l'Effort predetto e quello reale.
 - Questi parametri vengono aggiunti al modello come cost driver.

Regression based models

- $\text{Effort} = a + b \times \text{Size}$
 - Effort è lo sforzo in mesi persona (PM)
 - a, b sono costanti derivate empiricamente
 - Size è il fattore di costo primario (e.g., LOC, FP, o CFP)



Constraint models

- I Constraint model mostrano la relazione nel tempo tra due o più parametri di sforzo, durata o livello di personale (Fenton, 1997).
- Il modello SLIM di Putnam è un esempio di Constraint model: aggiunge la dimensione temporale allo sforzo rispetto al grafico delle dimensioni.

COCOMO

- COCOMO (Constructive Cost Model) (Boehm, 1981)
- E' un modello basato su input relativi alla dimensione del sistema e una serie di fattori di costo che influenzano la produttività.
- La versione aggiornata, denominata COCOMO II, tiene conto dei cambiamenti nelle tecnologie, incluso software OO, il software sviluppato con modelli a spirali ed evolutivi, il riuso del software e usando componenti off-the-shelf.

COCOMO

- La versione originale è una collezione di tre modelli:
 - Basic model applicato all'inizio del progetto
 - Intermediate model applicato dopo l'acquisizione dei requisiti
 - Advanced model applicato dopo il completamento della progettazione
- Tutti e tre i modelli assumono la forma:

$$E = a S^b * EAF$$

$$T_{dev} = c E^d$$

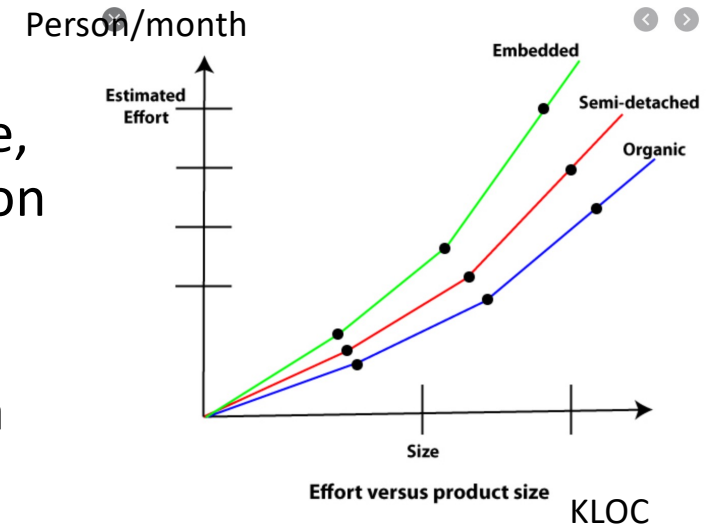
dove

- E è lo sforzo in mesi persona
- T_{dev} è il tempo di sviluppo
- S è la dimensione misurata in KLOC
- EAF è un fattore di regolazione dello sforzo (1 nel modello Base)
- I fattori a, b, c e d dipendono dalla modalità di sviluppo.

COCOMO

COCOMO ha tre modalità di sviluppo:

- Organic mode: piccoli team, ambiente familiare, applicazioni ben comprese, nessun requisito non funzionale difficile (FACILE)
- Semi-detached mode: il team di progetto potrebbe avere un mix di esperienze, il sistema potrebbe avere vincoli non funzionali più significativi, l'organizzazione potrebbe avere meno familiarità con l'applicazione (HARDER)
- Embedded mode: sistemi hardware / software, vincoli stretti, insolito per il team che ha una profonda esperienza applicativa (HARD)



Basic

- Il modello Basic calcola lo sforzo (persona-mese) in funzione della dimensione del programma. L'equazione di base è:

$$E_{\text{nom}} = a (\text{KLOC})^b$$

$$\text{EAF} = 1$$

- I fattori a e b per il modello Base sono:
 - Organic: a=2.4 e b=1.05
 - Semi-detached: a=3.0 e b=1.12
 - Embedded: a=3.6 e b=1.2
-
- T_{dev} non è definito

Intermediate

- Il modello intermediate calcola lo sforzo in funzione della dimensione del programma e di un insieme di fattori di costo. L'equazione è:

$$E = a (\text{KLOC})^b * \text{EAF}$$

$$T = cE_{\text{nom}}^d$$

- I fattori a, b, c e d per il modello intermediate sono:
- Organic: a=2.4 b=1.05 c=2.5 d=0.38
- Semi-detached: a=3.0 b=1.12 c=2.5 d=0.35
- Embedded: a=3.6 b=1.2 c=2.5 d=0.32
- Il fattore di aggiustamento dello sforzo (EAF) è calcolato utilizzando 15 cost driver (*caratteristiche relative al progetto*)

Intermediate

- I cost driver sono raggruppati in 4 categorie: *prodotto*, *piattaforma*, *personale* e *progetto*.
- Ogni costo driver è valutato su una scala ordinale di 6 punti che va da importanza molto bassa a extra alta
 - Extra high, Very high, High, Nominal, Low, Very low
- Sulla base della valutazione, viene determinato il *moltiplicatore dello sforzo* (EM) (Boehm, 1981). Il prodotto di tutti i moltiplicatori dello sforzo è l'EAF

$$E = E_{\text{nom}} * \text{EAF}$$

$$\text{EAF} = \text{EM}_1 * \text{EM}_2 * \dots * \text{EM}_{15}$$

Intermediate

ProductFactors

- Affidabilità (RELY)
- Dati (DATA)
- Complessità (CPLX)

Personnelfactors

- Capacità di analisi (ACAP)
- Capacità del programma (PCAP)
- Esperienza applicativa (APEX)
- Esperienza della piattaforma (PLEX)
- Esperienza linguaggi e tool (LTEX)
- Continuità del personale (PCON)

PlatformFactors

- Vincolo di tempo (TIME)
- Vincolo di archiviazione (STOR)
- Volatilità della piattaforma (PVOL)

ProjectFactors

- Tool software (TOOL)
- Sviluppo multisito (SITE)
- Schedule richiesto (SCED)

Intermediate

ProductFactors

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
RELY	Required software reliability	0.75	0.88	1.00	1.15	1.40	-
DATA	Database size	-	0.94	1.00	1.08	1.16	-
CPLX	Product complexity	0.70	0.85	1.00	1.15	1.30	1.65

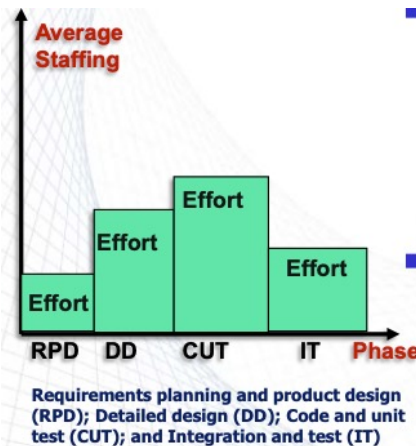
Intermediate

- Esempio:
 - affidabilità software richiesta (RELY)
 - Misura in cui il software deve eseguire le proprie funzioni previste per un periodo di tempo.
 - Chiedere: qual è l'effetto di un errore del software?

Very Low	Low	Nominal	High	Very High	Extra High
slight inconvenience	low, easily recoverable losses	moderate, easily recoverable losses	high financial loss	risk to human life	
0.75	0.88	1.00	1.15	1.40	

Advanced

- Il modello Advanced calcola lo sforzo in funzione della dimensione del programma e di una serie di cost driver ***pesati in base a ciascuna fase del ciclo di vita del software***.
- Il modello Advanced applica il modello Intermediate a livello di componente, e poi viene utilizzato un approccio basato sulle fasi per consolidare la previsione



Cost Driver	Rating	RPD	DD	CUT	IT
ACAP (Analyst capability)	Very Low	1.80	1.35	1.35	1.50
	Low	0.85	0.85	0.85	1.20
	Nominal	1.00	1.00	1.00	1.00
	High	0.75	0.90	0.90	0.85
	Very High	0.55	0.75	0.75	0.70

ACAP Intermediate
1.46 – Very Low
1.19 – Low
1.00 – Nominal
0.86 – High
0.71 – Very High

COCOMO II

- Comprende una serie di modelli a tre stadi:
- Le prime fasi coinvolgeranno generalmente la prototipazione, utilizzando le ***Application Composition*** model capabilities. (sostituisce il COCOMO basic)
- Le fasi successive implicheranno generalmente l'esplorazione di alternative architetture o strategie di sviluppo incrementale. Per supportare queste attività, COCOMO II fornisce un modello di stima iniziale chiamato ***Early Design model***. (sostituisce il COCOMO Intermediate)
- Una volta che il progetto è pronto per sviluppare e sostenere un sistema in campo, dovrebbe avere un'architettura, che fornisce informazioni più accurate sui cost driver e consente stime dei costi più accurate. A supporto di questa fase, COCOMO II fornisce il ***Post-Architecture model***. (sostituisce il COCOMO Advanced)

Application Composition model

- L'**Application Composition model** viene utilizzato nelle prime fasi di sviluppo e la prototipazione per risolvere potenziali problemi ad alto rischio come interfacce utente, interazione software / sistema, performance o maturità tecnologica.
- **Object points** vengono utilizzati per il dimensionamento anziché la metrica LOC tradizionale.
 - $E = OP / PROD$

dove

- OP : Object points
- PROD: tasso di produttività

<i>Developers' experience and capability</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>
PROD	4	7	13	25	50

Early Design Model

- Il modello **Early Design** viene utilizzato per valutare architetture software / di sistema alternative e functional concepts. Per il dimensionamento viene utilizzato il conteggio dei Function Points (senza VAF). Questo valore viene convertito in KLOC.
- L'equazione del modello è:
$$E = 2,45 \times \text{KLOC} \times \text{EAF}$$
- Il fattore di aggiustamento dello sforzo (EAF) è calcolato come nel modello COCOMO originale utilizzando 7 cost drivers

Early Design Model

- EAF ottenuto combinando i 17 cost driver del Post-Architecture Model

	<i>Cost Driver</i>	<i>Description</i>	<i>Combined Post-Architecture Cost Drivers</i>
1	RCPX	Product reliability and complexity	RELY, DATA, CPLX, DOCU
2	RUSE	Required reuse	RUSE
3	PDIF	Platform difficulty	TIME, STOR, PVOL
4	PERS	Personnel capability	ACAP, PCAP, PCON
5	PREX	Personnel experience	AEXP, PEXP, LTEX
6	FCIL	Facilities	TOOL, SITE
7	SCED	Schedule	SCED

Post-Architecture Model

- Il Post-Architecture Model viene utilizzato durante lo sviluppo e la manutenzione di un prodotto.
- FP o LOC possono essere utilizzati per il dimensionamento, con modificatori per il riutilizzo.
- Il Post-Architecture Model include una serie di **17 cost driver (EAF)** e **5 fattori di scala (SF)** che determinano la componente di scalabilità del progetto.
- L'equazione è:

$$E = 2,45 * (KLOC \times C)^b * EAF$$

$$b = 0,91 + 0,01 * (SF_1 + SF_2 + \dots + SF_5) \quad 0.91 \leq b \leq 1.23$$

Scale Factor

Scale Factor SF		Very Low	Low	Nominal	High	Very High	Extra High
Precedentedness Degree to which system is new and past experience applies	PREC	6.20	4.96	3.72	2.48	1.24	0.00
Development/Flexibility Need to conform to specified requirements	FLEX	5.07	4.05	3.04	2.03	1.01	0.00
Architecture/Risk Resolution Degree of design thoroughness and risk elimination	RESL	7.07	5.65	4.24	2.83	1.41	0.00
Team Cohesion Need to synchronize stakeholders and minimize conflict	TEAM	5.48	4.38	3.29	2.19	1.10	0.00
Process Maturity SEI CMM process maturity rating	PMAT	7.80	6.24	4.68	3.12	1.56	0.00

PREC e FLEX

Feature	Very Low	Nominal / High	Extra High
Precedentedness			
Organizational understanding of product objectives	General	Considerable	Thorough
Experience in working with related software systems	Moderate	Considerable	Extensive
Concurrent development of associated new hardware and operational procedures	Extensive	Moderate	Some
Need for innovative data processing architectures, algorithms	Considerable	Some	Minimal
Development Flexibility			
Need for software conformance with pre-established requirements	Full	Considerable	Basic
Need for software conformance with external interface specifications	Full	Considerable	Basic
Premium on early completion	High	Medium	Low

RESL

Characteristic	Very Low	Low	Nominal	High	Very High	Extra High
Risk Management Plan identifies all critical risk items, establishes milestones for resolving them by PDR.	None	Little	Some	Generally	Mostly	Fully
Schedule, budget, and internal milestones through PDR compatible with Risk Management Plan	None	Little	Some	Generally	Mostly	Fully
Percent of development schedule devoted to establishing architecture, given general product objectives	5	10	17	25	33	40
Percent of required top software architects available to project	20	40	60	80	100	120
Tool support available for resolving risk items, developing and verifying architectural specs	None	Little	Some	Good	Strong	Full
Level of uncertainty in Key architecture drivers: mission, user interface, COTS, hardware, technology, performance.	Extreme	Significant	Considerable	Some	Little	Very Little
Number and criticality of risk items	> 10 Critical	5-10 Critical	2-4 Critical	1 Critical	> 5 Non-Critical	< 5 Non-Critical

TEAM

Characteristic	Very Low	Low	Nominal	High	Very High	Extra High
Consistency of stakeholder objectives and cultures	Little	Some	Basic	Considerable	Strong	Full
Ability, willingness of stakeholders to accommodate other stakeholders' objectives	Little	Some	Basic	Considerable	Strong	Full
Experience of stakeholders in operating as a team	None	Little	Little	Basic	Considerable	Extensive
Stakeholder teambuilding to achieve shared vision and commitments	None	Little	Little	Basic	Considerable	Extensive

PMAT

- Elaborazione della scala di valutazione PMAT:
 - Due metodi basati sul Capability Maturity Model (CMM)
 - Metodo 1: Overall Maturity Level (CMM Level 1 through 5)
 - Metodo 2: Key Process Areas (KPA)
- KPA: Decidere la percentuale di conformità per ciascuno dei KPA tramite una media basata sul giudizio tra gli obiettivi per tutte le 18 Key Process Areas.

$$5 - \left[\sum_{i=1}^{18} \left(\frac{KPA\%_i}{100} \times \frac{5}{18} \right) \right]$$

PMAT: KPA

[illegible]

Cost driver

ProductFactors

- Affidabilità (RELY)
- Dati (DATA)
- Complessità (CPLX)
- Riutilizzabilità richiesta (RUSE)
- Documentazione (DOCU)

PersonneFactors

- Capacità di analisi (ACAP)
- Capacità del programma (PCAP)
- Esperienza applicativa (APEX)
- Esperienza della piattaforma (PLEX)
- Esperienza linguaggi e tool (LTEX)
- Continuità del personale (PCON)

PlatformFactors

- Vincolo di tempo (TIME)
- Vincolo di archiviazione (STOR)
- Volatilità della piattaforma (PVOL)

ProjectFactors

- Tool software (TOOL)
- Sviluppo multisito (SITE)
- Schedule richiesto (SCED)

Post-Architecture cost drivers

Product

	<i>Description</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
RELY	Required software reliability	0.82	0.92	1.00	1.10	1.26	-
DATA	Database size	-	0.90	1.00	1.14	1.28	-
CPLX	Product complexity	0.73	0.87	1.00	1.17	1.34	1.74
RUSE	Required reusability	-	0.95	1.00	1.07	1.15	1.24
DOCU	Documentation	0.81	0.91	1.00	1.11	1.23	

SLIM (Cenni)

- Gestione del ciclo di vita del software (SLIM)
- SLIM è un sistema automatizzato di determinazione dei costi basato sul Modello di Rayleigh-Putnam

$$\text{Work product} = \text{process productivity} \times \text{Effort}^a \times \text{Time}^b$$

- SLIM applica il modello del software Putnam, la programmazione lineare, la simulazione statistica e le tecniche di valutazione e revisione del programma (PERT) per derivare le stime dei progetti software.

SLIM (Cenni)

Model: equation

$$size = C \times B^{1/3} \times T^{4/3}$$

$$B = \left(\frac{1}{T^4} \right) \left(\frac{size}{C} \right)^3$$

$$E = 0.3935B$$

- C: parametro della produttività del processo
- B: impegno totale (personale-anni) - dipende dalle dimensioni e dal tempo!
- T: tempo di sviluppo (anni)
- size: stima della taglia in LOC
- E: sforzo totale al momento della consegna (picco della curva di Rayleigh)

SLIM (Cenni)

- Il sistema consente a un pianificatore software di eseguire le seguenti funzioni in una sessione interattiva:
 - Stima i parametri per i nuovi progetti.
 - Calibrare l'ambiente di sviluppo software, da interpretare i dati storici forniti dal progettista.
 - Genera piani di soluzione con una specifica probabilità di successo in base alle stime.
- La famiglia di prodotti SLIM si basa sul concetto di indice di produttività (PI). PI è una misura di alto livello della capacità di un team di sviluppo di produrre software.
- PI viene calcolato in base alle dimensioni, al tempo, all'impegno e alla complessità dell'applicazione. (riflette il fattore C nella formula)

SLIM (Cenni)

- SLIM funziona abbastanza bene per sistemi molto grandi, ma sovrastima seriamente lo sforzo per sistemi medi e piccoli.
- Alcune versioni di SLIM utilizzano un cosiddetto fattore di abilità β , che è un termine scalabile ed è una funzione della dimensione del progetto.

$$\text{size} = C \times (B / \beta)^{1/3} \times T^{4/3}$$

Size (SLOC)	β
5-15K	0.16
20K	0.18
30K	0.28
40K	0.34
50K	0.37
>70K	0.39

Other: Bottom-up estimation

- Dal livello di sistema più basso si stima il costo (e la schedulazione) di ogni componente.
- Tutti i costi ottenuti sono usati per ottenere la stima finale.
- Vantaggio: accurato se il sistema è stato progettato in dettaglio
- Svantaggio: sottostima i costi di attività come integrazione e documentazione (a livello di sistema)

Other: Top-down estimation

- Una stima del costo (e della schedulazione) considerando le funzionalità dell'intero prodotto e il modo in cui tali funzionalità sono ottenute tramite interazioni di sotto-funzioni
- La stima dei costi ottenuta a partire dalle funzionalità e non dalle componenti che implementano tali funzionalità.
- Vantaggio: considera i costi quali integrazione e della documentazione
- Svantaggio: sottostima i costi relativi alla risoluzione di problemi tecnici difficili di basso livello

Considerazioni finali

- **I modelli algoritmici esistenti possono essere applicati a un'ampia gamma di progetti?**
- Si suggerisce che un modello sia accettabile se il 75% dei valori previsti rientra nel 25% dei loro valori effettivi (Fenton, 1997).
- Sfortunatamente la maggior parte dei modelli è insufficiente in base a questo criterio.

Critiche: struttura

- Sebbene la maggior parte dei ricercatori e dei professionisti siano concordi sul fatto che la dimensione è il determinante principale dello sforzo, l'esatta relazione tra la dimensione e lo sforzo non è chiara (Fenton, 1997).
- C'è poco consenso sull'effetto di ridurre o estendere la durata.
 - I cost driver nel modello COCOMO di B. Boehm presuppone che l'aumento o la diminuzione della durata aumenti lo sforzo del progetto.
 - Il modello SLIM di Putnam implica che la diminuzione della durata aumenta lo sforzo e l'aumento della durata diminuisce lo sforzo.
 - Altri studi hanno dimostrato che la diminuzione della durata diminuisce lo sforzo, contraddicendo entrambi i modelli.
- La maggior parte dei modelli funziona bene negli ambienti per i quali sono stati derivati.

Critiche: size

- La maggior parte dei modelli richiede una stima delle dimensioni del prodotto. Tuttavia, le dimensioni sono difficili da prevedere all'inizio del ciclo di vita dello sviluppo.
- Sebbene i Function Points e Object Points possano essere utilizzati nelle prime fasi del ciclo di vita, queste misure sono "estremamente" soggettive.
- Le stime della size possono anche essere molto imprecise. I metodi di stima e raccolta dei dati devono essere coerenti per garantire una previsione accurata delle dimensioni del prodotto. A meno che le metriche delle dimensioni utilizzate nel modello non siano le stesse utilizzate nella pratica, il modello non produrrà risultati accurati (Fenton, 1997).

Critiche: complessità

- Molti modelli includono fattori di aggiustamento per tenere conto di particolari progetti e determinate pratiche di sviluppo.
- Molti dei fattori di aggiustamento, come i cost driver, si influenzano a vicenda, determinando un'enfasi eccessiva di determinati attributi. Anche i cost driver sono estremamente soggettivi (Fenton, 1997).
- Anche il calcolo del fattore di aggiustamento è spesso complicato. Ad esempio, anche il calcolo di EAF per COCOMO dettagliato può essere alquanto complesso, poiché è distribuito tra le fasi del ciclo di vita del software.

Conclusioni

- La stima del costo del software dovrebbe essere parte integrante del processo di sviluppo del software.
- I modelli di stima dei costi possono essere utilizzati per rappresentare la relazione tra lo sforzo e un fattore di costo primario come la dimensione.
- I cost driver sono utilizzati per adeguare la stima preliminare fornita dal fattore di costo primario.
- I modelli di stima dei costi soffrono di alcuni problemi comuni:
 - I modelli si basano su risultati empirici piuttosto che sulla teoria.
 - I modelli sono spesso complessi e si basano molto sulla stima delle dimensioni.